

Week 10 Report

Fix Suggestion Generation

MADDELA RAJEEV
Roll No: 24CSB0B40
Compiler Design (CS1202)

1 Introduction

In Week 10, the focus of the project shifted from error classification to generating meaningful fix suggestions for compiler errors. While the system developed in previous weeks could successfully classify syntax and parsing errors using machine learning, it lacked the ability to guide users on how to resolve those errors.

The primary goal of this phase is to enhance usability by providing human-readable explanations and actionable correction steps. This transforms the system from a diagnostic tool into an intelligent assistant for beginner programmers.

2 Objective

The main objectives of Week 10 are:

- To map classified error types to appropriate correction strategies
- To generate clear and beginner-friendly explanations
- To provide step-by-step fix suggestions
- To include code examples demonstrating correct usage

3 System Design

The fix suggestion module is designed as a rule-based system integrated with the machine learning classifier. The workflow is as follows:

1. Compiler generates raw error message
2. Machine learning model classifies error type
3. Classified label is passed to suggestion engine

4. Corresponding explanation and fix are retrieved
5. Enhanced output is displayed to the user

This hybrid approach combines the strengths of machine learning (classification) and rule-based systems (explanation generation).

4 Implementation

The fix suggestion engine is implemented using a dictionary-based mapping approach in Python. Each error category is associated with:

- Explanation of the error
- List of fix steps
- Example showing incorrect and corrected code

4.1 Example

Error Type: Missing Semicolon

- Explanation: A statement must end with a semicolon.
- Fix:
 1. Go to the specified line
 2. Add a semicolon at the end

Example Code:

Wrong:

```
int x = 10
```

Correct:

```
int x = 10;
```

5 Output Enhancement

The system now produces structured output containing:

- Original compiler error message
- Predicted error category

- Explanation of the error
- Step-by-step fix suggestions
- Example code (wrong vs correct)
- Confidence score of prediction

This significantly improves user understanding and debugging efficiency.

6 Advantages

- Beginner-friendly explanations
- Reduced debugging time
- Clear understanding of errors
- Educational value through examples
- Completes the AI-assisted error diagnosis pipeline

7 Conclusion

Week 10 marks a crucial step in transforming the system into a complete AI-assisted compiler error diagnosis tool. By integrating fix suggestions with classification, the system now provides both identification and resolution of errors.

This enhancement aligns with the project goal of improving programming learning experiences and reducing confusion caused by traditional compiler messages.

8 Future Work

Future improvements may include:

- Context-aware suggestions using AST
- Automatic code correction
- Support for additional programming languages
- Integration with IDEs